



School of Computer Science

COMP47470: Big Data Programming

Assignment 2

Neo4j NOSQL DBMS

Module Coordinator	Ravi Reddy Manumachu
Email	ravi.manumachu@ucd.ie

Component 1	2
Loading Flights	2
Checksum Queries Before Attempting the Assignment	2
Problems to Solve	4
Component 2	5
Refresh and Reload Neo4j	5
Checksum Queries Before Attempting the Assignment	6
Problems to Solve	8

This assignment is about Neo4j NoSQL DBMS and Cypher Programming Language.

You can complete this assignment using Neo4j installation on your own laptop/desktop and submit the solutions in a **solution.txt** or **solutions.pdf** file.

The assignment carries 10 points (10% weight). It contains two components.

Component 1

In the first component, you will work on a **flights Property Graph**. The **Neo4j database dump** of this file is given in a zip file. Unzip the file before loading the dump.

Loading Flights

Note the following instructions are for Ubuntu Linux distributions only. Use corresponding instructions if you are using Windows or Mac to solve this assignment.

Load the database using the **neo4j-admin** tool from the movies directory. Before that, you must stop the neo4j service.

```
$ sudo systemctl stop neo4j
```

```
$ cp neo4j.dump /tmp/
```

```
$ sudo -u neo4j neo4j-admin database load neo4j --from-path=/tmp --overwrite-destination=true
```

Done: 36 files, 264.7MiB processed in 0.330 seconds.

Start the neo4j service and run the queries in **cypher-shell**.

```
$ sudo systemctl start neo4j
```

```
$ cypher-shell -u neo4j -p neo4j123
```

Checksum Queries Before Attempting the Assignment

Run the following queries and check the output provided with what you get to make sure the database is properly loaded.

1. List all node labels.

```
CALL db.labels();
```

```
neo4j@neo4j> CALL db.labels();
+-----+
| label |
+-----+
| "Airport" |
| "Airline" |
+-----+

2 rows
ready to start consuming query after 61 ms, results consumed after another 4 ms
```

2. List all relationship types.

```
CALL db.relationshipTypes();
```

```
neo4j@neo4j> CALL db.relationshipTypes();
+-----+
| relationshipType |
+-----+
| "ROUTE"          |
+-----+

1 row
ready to start consuming query after 46 ms, results consumed after another 1 ms
```

3. List all property keys to see what attributes are stored on nodes and relationships.

CALL db.propertyKeys();

```
neo4j@neo4j> CALL db.propertyKeys();
+-----+
| propertyKey |
+-----+
| "country"   |
| "iata"      |
| "city"      |
| "latitude"  |
| "name"      |
| "id"        |
| "longitude" |
| "icao"       |
| "airline"   |
+-----+

9 rows
ready to start consuming query after 47 ms, results consumed after another 2 ms
```

4. Count nodes per label to see how many of each type of node exist.

MATCH (n)

RETURN labels(n) AS label, count(*) AS count

ORDER BY count DESC;

```
neo4j@neo4j> MATCH (n)
              RETURN labels(n) AS label, count(*) AS count
              ORDER BY count DESC;
+-----+
| label      | count |
+-----+
| ["Airport"] | 7698  |
| ["Airline"] | 6162  |
+-----+

2 rows
ready to start consuming query after 328 ms, results consumed after another 53 ms
```

5. Count relationships per type to find out how many of each kind of relationship exist, revealing which connections dominate the graph.

MATCH ()-[r]->()

RETURN type(r) AS relationshipType, count(*) AS count

ORDER BY count DESC;

```
neo4j@neo4j> MATCH ()-[r]->()
RETURN type(r) AS relationshipType, count(*) AS count
ORDER BY count DESC;
+-----+
| relationshipType | count |
+-----+
| "ROUTE"         | 66924 |
+-----+

1 row
ready to start consuming query after 154 ms, results consumed after another 77 ms
```

If all the above checks pass, then you are ready to attempt the component 1.

Problems to Solve

Problem 1).

Find the **total number of distinct destinations** reachable from **Dublin** in a single hop. The International Air Transport Association (IATA) code for Dublin is **DUB**.

The query should display only the destination count.

What is the Cypher query statement and the output from the query execution? (1 point)

Problem 2).

Find the **country** that has the **maximum number of outbound routes** in the dataset.

The query should display both the **country** and the **number of outbound routes**.

What is the Cypher query statement and the output from the query execution? (1 point)

Problem 3).

Find the **top airline** by the number of routes it operates in the dataset.

The query should display the **name of the airline** and the **count of routes**.

What is the Cypher query statement and the output from the query execution? (1 point)

Problem 4).

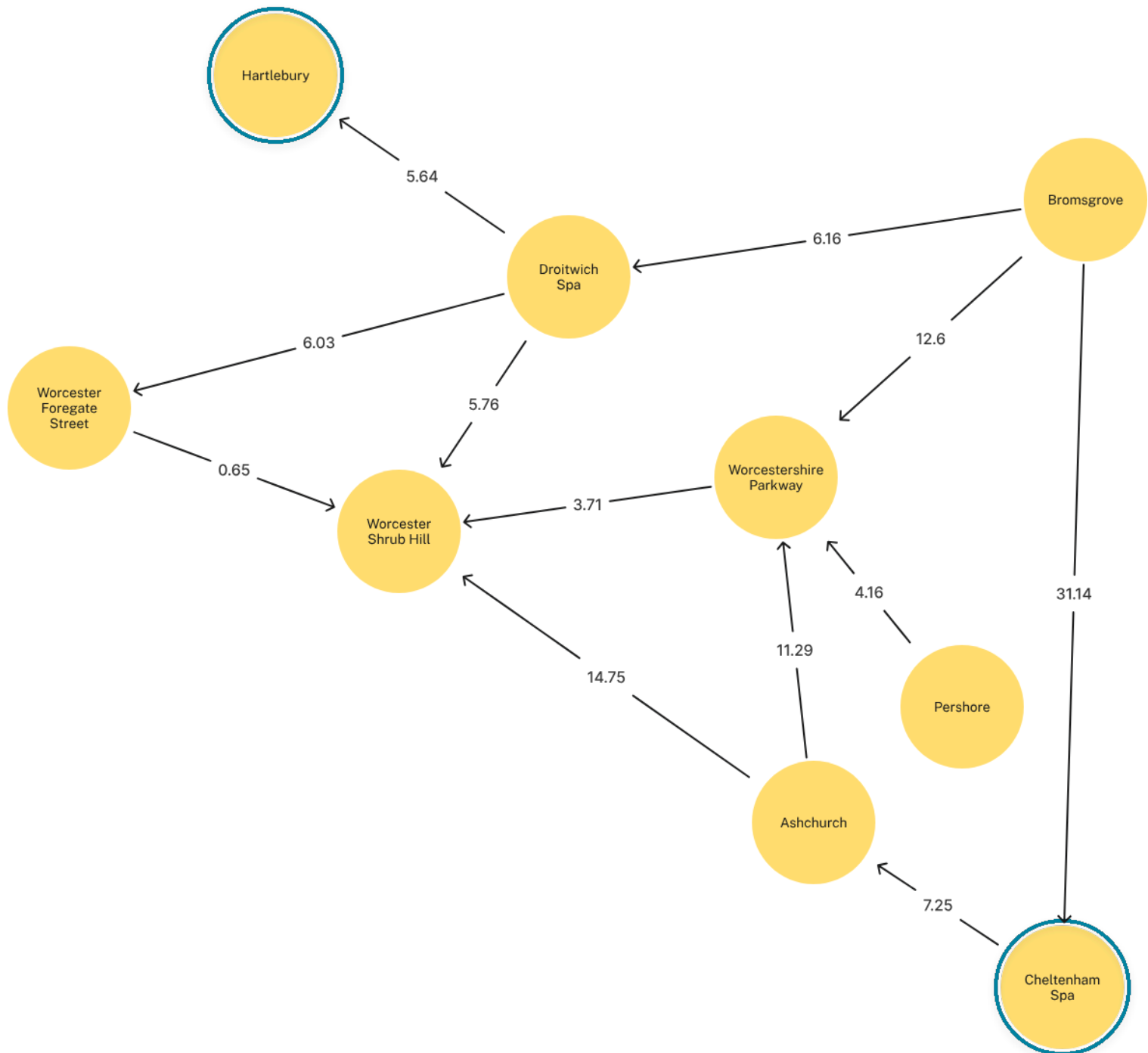
Find the **airline** that operates the **most number of routes** between **distinct countries**.

The query should display the **name of the airline** only.

What is the Cypher query statement and the output from the query execution? (2 points)

Component 2

In this component, you will work with the following property graph for path finding.



Refresh and Reload Neo4j

```
$ cypher-shell -u neo4j -p neo4j123 "MATCH (n) DETACH DELETE n;"
```

ready to start consuming query after 463 ms, results consumed after another 0 ms

Deleted 13860 nodes, deleted 66924 relationships

```
$ sudo systemctl stop neo4j
```

```
$ sudo systemctl start neo4j
```

```
$ sudo rm -rf /var/lib/neo4j/data/databases/neo4j
```

```
$ sudo rm -rf /var/lib/neo4j/data/transactions/neo4j
```

Run the following load script queries in **cypher-shell**.

```
CREATE (asc:Station {name:"Ashchurch"}),  
      (bmv:Station {name:"Bromsgrove"}),
```

```

(cnm:Station {name:"Cheltenham Spa"}),
(dtw:Station {name:"Droitwich Spa"}),
(hby:Station {name:"Hartlebury"}),
(psh:Station {name:"Persnore"}),
(wop:Station {name:"Worcestershire Parkway"}),
(wof:Station {name:"Worcester Foregate Street"}),
(wos:Station {name:"Worcester Shrub Hill"})
CREATE (asc)-[:LINK {distance: 7.25}]->(cnm),
(asc)-[:LINK {distance: 11.29}]->(wop),
(asc)-[:LINK {distance: 14.75}]->(wos),
(bmv)-[:LINK {distance: 31.14}]->(cnm),
(bmv)-[:LINK {distance: 6.16}]->(dtw),
(bmv)-[:LINK {distance: 12.6}]->(wop),
(dtw)-[:LINK {distance: 5.64}]->(hby),
(dtw)-[:LINK {distance: 6.03}]->(wof),
(dtw)-[:LINK {distance: 5.76}]->(wos),
(psh)-[:LINK {distance: 4.16}]->(wop),
(wop)-[:LINK {distance: 3.71}]->(wos),
(wof)-[:LINK {distance: 0.65}]->(wos);

```

```

neo4j@neo4j> CREATE (asc:Station {name:"Ashchurch"}),
                (bmv:Station {name:"Bromsgrove"}),
                (cnm:Station {name:"Cheltenham Spa"}),
                (dtw:Station {name:"Droitwich Spa"}),
                (hby:Station {name:"Hartlebury"}),
                (psh:Station {name:"Persnore"}),
                (wop:Station {name:"Worcestershire Parkway"}),
                (wof:Station {name:"Worcester Foregate Street"}),
                (wos:Station {name:"Worcester Shrub Hill"})
CREATE (asc)-[:LINK {distance: 7.25}]->(cnm),
(asc)-[:LINK {distance: 11.29}]->(wop),
(asc)-[:LINK {distance: 14.75}]->(wos),
(bmv)-[:LINK {distance: 31.14}]->(cnm),
(bmv)-[:LINK {distance: 6.16}]->(dtw),
(bmv)-[:LINK {distance: 12.6}]->(wop),
(dtw)-[:LINK {distance: 5.64}]->(hby),
(dtw)-[:LINK {distance: 6.03}]->(wof),
(dtw)-[:LINK {distance: 5.76}]->(wos),
(psh)-[:LINK {distance: 4.16}]->(wop),
(wop)-[:LINK {distance: 3.71}]->(wos),
(wof)-[:LINK {distance: 0.65}]->(wos);

0 rows
ready to start consuming query after 602 ms, results consumed after another 0 ms
Created 9 nodes, created 12 relationships, set 21 properties, added 9 labels

```

Checksum Queries Before Attempting the Assignment

Run the following queries and check the output provided with what you get to make sure the property graph is properly created.

1. List all node labels.

CALL db.labels();

```

neo4j@neo4j> CALL db.labels();
+-----+
| label |
+-----+
| "Station" |
+-----+

1 row
ready to start consuming query after 40 ms, results consumed after another 4 ms

```

2. List all relationship types.

CALL db.relationshipTypes();

```
neo4j@neo4j> CALL db.relationshipTypes();
+-----+
| relationshipType |
+-----+
| "LINK"           |
+-----+

1 row
ready to start consuming query after 32 ms, results consumed after another 2 ms
```

3. List all property keys to see what attributes are stored on nodes and relationships.

CALL db.propertyKeys();

```
neo4j@neo4j> CALL db.propertyKeys();
+-----+
| propertyKey |
+-----+
| "name"      |
| "distance"  |
+-----+

2 rows
ready to start consuming query after 33 ms, results consumed after another 2 ms
```

4. Count nodes per label to see how many of each type of node exist.

MATCH (n)

RETURN labels(n) AS label, count(*) AS count

ORDER BY count DESC;

```
neo4j@neo4j> MATCH (n)
              RETURN labels(n) AS label, count(*) AS count
              ORDER BY count DESC;
+-----+
| label      | count |
+-----+
| ["Station"] | 9     |
+-----+

1 row
ready to start consuming query after 278 ms, results consumed after another 9 ms
```

5. Count relationships per type to find out how many of each kind of relationship exist, revealing which connections dominate the graph.

MATCH ()-[r]->()

RETURN type(r) AS relationshipType, count(*) AS count

ORDER BY count DESC;

```
neo4j@neo4j> MATCH ()-[r]->()
              RETURN type(r) AS relationshipType, count(*) AS count
              ORDER BY count DESC;
+-----+
| relationshipType | count |
+-----+
| "LINK"           | 12    |
+-----+

1 row
ready to start consuming query after 118 ms, results consumed after another 3 ms
```

If all the above checks pass, then you are ready to attempt component 2.

Problems to Solve

Problem 1).

You will write a path-finding query to find **the shortest route** from **Ashchurch** to **Worcester Shrub Hill**.

The query should display the **route** and **the number of stops** given by the **length of the path**.

What is the Cypher query statement and the output from the query execution? (2 points)

Problem 2).

You will write a path-finding query to find **all equally shortest routes** from **Worcester Shrub Hill** to **Bromsgrove**.

The query should display the **routes** and **the number of stops** for each route..

What is the Cypher query statement and the output from the query execution? (1 point)

Problem 3).

You will write a path-finding query to find **any two possible routes** from **Ashchurch** to **Worcester Shrub Hill**.

The query should display the **routes** and **the number of stops** given by the **length of the path**.

What is the Cypher query statement and the output from the query execution? (2 points)